> *In addition, Serverless Functions have a maximum duration, meaning that it isn't possible to stream indefinitely. Edge Functions do not have a maximum duration, but you must send an initial response within 25 seconds. You can continue streaming a response beyond that time.*

So choosing an appropriate runtime will affect the reliability of your services. If you configure and respect the streaming limits, then streaming will work as expected.

Still, the best strategy is to add structured logging in key places on your app and then review and filter those logs to detect what is going wrong. Is it your code that is misbehaving, or is it the AI provider that is not responding to requests? Is your deployment configuration missing some important elements so that at run time you encounter critical errors? All those questions can be answered if you have access to detailed error reporting or relevant error logs.

To implement structured logging in a Next.js application, I suggest starting with a dedicated logging library such as the Pino logging library. In any file where you want to log messages, import the logger and create a child logger as needed. This allows you to set attributes that will appear on every log line, making it easier to identify the source of logs. The following listing shows a simple logger you can use.

Listing 8.1 Creating a Pino logger instance

```
import pino from 'pino';

export const logger =
  process.env.NODE_ENV === 'production'
    ? pino({ level: 'warn' })
    : pino({
        transport: {.
          target: 'pino-pretty',
          options: {
            colorize: true,
          },
        },
        level: 'debug',
      })
```

Imports the Pino logging library for structured logging

Uses 'pino-pretty' for formatted console output during development

For your convenience, I've included several logger calls in both the client and server actions codebase, which you can inspect when running a few queries (see figure 8.3).

Logging in general is universal and can help you uncover and understand problems that stem from various aspects of application development. It is highly recommended to include log statements as a first step in key areas of the code (such as API responses, before and after state changes) that you are working on.

8.1.3 Handling state management

State management problems in Next.js happen because there are often complex interactions between client and server state, component lifecycles, and data flow patterns. For these, use *logging* and remember to *keep the AI logic separate from UI components.*